User manual for atorvi - ATomic ORbitals VIsualization package

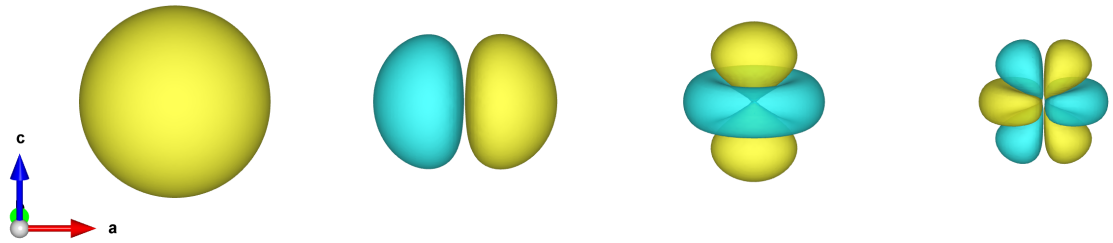

Table of contents

Introduction

`atorvi` is a Python package for creating three-dimensional visualizations of atomic orbitals in crystalline materials. These visualizations can be used in research publications, educational materials, and scientific analysis.

Common applications in condensed matter physics and solid-state chemistry include:

1. **Magnetic Ordering**: Visualize orbital overlap between magnetic atoms in ferromagnetic and antiferromagnetic materials to understand exchange interactions between neighboring sites.
2. **Chemical Bonding**: Examine hybrid orbitals (sp^2, sp^3) in semiconductors and insulators to understand covalent bonding and electron sharing between atoms.
3. **Electronic Correlations**: Study electron localization in strongly correlated systems like Mott insulators, complementing computational results from DFT+U and DMFT calculations.
4. **Band Structure Analysis**: Map the orbital contributions to electronic bands when analyzing band structures and density of states (DOS).
5. **Crystal Field Theory**: Explore how crystal fields affect d-orbital splitting in transition metal compounds, helping explain their electronic, magnetic and optical behavior.

The package generates files in XCrysDen format, enabling both interactive exploration and high-resolution image export.

Minimal theory: Atomic orbitals

The atomic orbital of the hydrogen-like atom with quantum numbers n and ℓ is calculated as:

$$\psi_{n\ell}(\mathbf{r}) = R_{n\ell}(r) X_{\ell c}(\mathbf{r}),$$

where $X_{\ell c}$ are the cubic harmonics.

The radial part of the atomic orbital is calculated as:

$$R_{n\ell}(r) = \sqrt{\left(\frac{2Z}{na_0}\right)^3 \frac{(n-\ell-1)!}{2n(n+\ell)!}} e^{-Zr/na_0} \left(\frac{2Zr}{na_0}\right)^{\ell} L_{n-\ell-1}^{(2\ell+1)}\left(\frac{2Zr}{na_0}\right),$$

where: $L_{n-\ell-1}^{(2\ell+1)}$ – are the generalized Laguerre polynomials, a_0 is the Bohr radius and Z is the screened nuclear charge. We use the effective nuclear charge by Clementi *et al.* to account the shielding effect of inner-shell electrons on outer-shell electrons, providing a more accurate representation of the potential energy experienced by electrons in multi-electron atoms during calculations.

`atorvi` always generates orbitals of the outermost shell for a given element.

Installation

To install **atorvi**, you can use `pip`:

```
pip install atorvi
```

Workflow

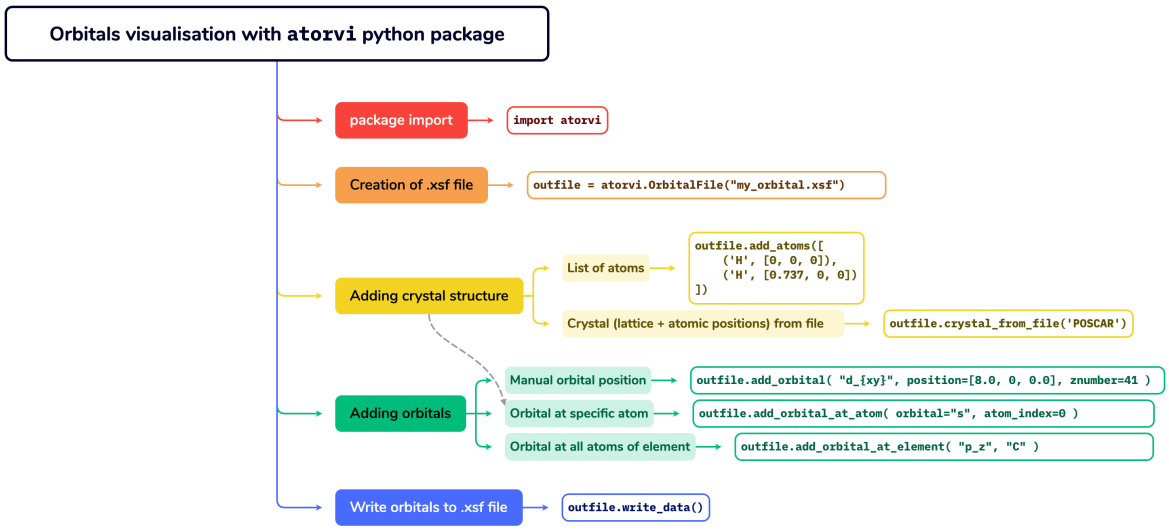

Figure 1: Workflow of atorvi package

Step 0: Import the package

In your python script or Jupyter notebook write

```
import atorvi
```

After the import, several useful objects are available for you, such as

```
atorvi.supported_orbitals,
atorvi.p_orbitals,
atorvi.d_orbitals,
atorvi.f_orbitals
```

Step 1: Create the output file in XCrysDen format

Create an instance of the `OrbitalFile` class, which will be used to write the `.xsf` file.

```
outfile = atorvi.OrbitalFile("my_orbital.xsf")
```

Step 2: Add atoms to the file

This step is optional. You can generate orbitals without adding atoms to the file.

You can add atoms to the file manually creating a non-periodic molecule, or you can read crystal structure from a (XSF/POSCAR/CIF) file.

2.1. Manual atoms addition `.add_atoms()` method accepts a list of tuples, where each tuple contains the atomic symbol and the coordinates of the atom. You can add atoms one-by-one or in a batch. Atomic coordinates are given in angstroms.

```
outfile.add_atoms([
    ('H', [0, 0, 0]),
    ('H', [0.737, 0, 0])
])
```

2.2. Reading atoms from a file `.crystal_from_file()` method accepts a path to the file with crystal structure. We use pymatgen to read the file, so this package should be installed in your environment with `pip install pymatgen` to use this method. Supported file formats are the same that pymatgen supports.

```
structure = outfile.crystal_from_file('./KCuF3_structure.xsf')
```

this method returns a `pymatgen.core.structure.Structure` object, which can be used to further manipulate the crystal structure in your script if necessary.

See example of reading structure from file in examples/structure_from_file/

Step 3: Add orbitals to the file

You can add to the file the following orbitals:

```
print(atorvi.supported_orbitals)
```

```
['s',
'p_z', 'p_x', 'p_y',
'd_{3z^2-r^2}', 'd_{xz}', 'd_{yz}', 'd_{x^2-y^2}', 'd_{xy}',
'f_{z^3}', 'f_{xz^2}', 'f_{yz^2}', 'f_{z(x^2-y^2)}', 'f_{xyz}',
'f_{x(x^2-3y^2)}', 'f_{y(3x^2-y^2)}']
```

3.1. Orbitals at arbitrary positions You can add an orbital at the arbitrary position using `.add_orbital(orbital, position, znumber, coeff)` method. This method accepts the following arguments:

```
"""
orbital : str
    The type of orbital (e.g., "s", "p_x", "d_{xy}", etc.).
position : list, optional
    The position of the orbital in angstroms (default is [0.0, 0.0, 0]).
znumber : int, optional
    The atomic number of the element (default is 8 i.e. Oxygen).
coeff : float, optional
    The coefficient of the orbital (default is 1.0).
"""
```

Example:

```
outfile.add_orbital("d_{3z^2-r^2}", position=[0, 0, 0], znumber=41)
```

See also examples/d_orbitals/ and examples/f_orbitals/ for more examples.

3.2. Orbitals at atoms positions If you have created some atoms at the Step 2, you can add an orbital at the position of the *i*-th atom using `.add_orbital_at_atom(orbital, atom_index, coeff)` method. This method accepts the following arguments:

```
"""
orbital : str
    The type of orbital (e.g., "s", "p_x", "d_{xy}", etc.).
atom_index : int
    The index of the atom in the system.
coeff : float, optional
    The coefficient of the orbital (default is 1.0).
"""
```

Example:

```
atoms = [
    ('Cu', [0.0,0.0,0.0]),
    ('Cu', [2.0,0.0,0.0]),
    ]

outfile.add_atoms(atoms)

outfile.add_orbital_at_atom('d_{xz}', 0)
outfile.add_orbital_at_atom('d_{yz}', 1)
```

See also examples/orbital_ordering for more examples.

3.3. Orbitals at all atoms of the same element You can add an orbital at all atoms of the same element using `.add_orbital_at_element(orbital, element, coeff)` method. This method accepts the following arguments:

```
"""
orbital : str
    The type of orbital (e.g., "s", "p_x", "d_{xy}", etc.).
element : str
    The symbol of the element.
coeff : float, optional
    The coefficient of the orbital (default is 1.0).
"""
```

Example:

```
outfile.add_orbital_at_element("p_z", "C")
```

See also examples/orbital_ordering/ for more examples.

3.4. Mixing (or hybridization) of atomic orbitals You can get hybridized (sp^2 or sp^3 or *etc*) orbitals combining the atomic orbitals with specific coefficients using the `coeff` parameter of the `.add_orbital*` methods. For example, the sp^3 hybridized orbital is: $\frac{1}{2}s + \frac{1}{2}p_x + \frac{1}{2}p_y + \frac{1}{2}p_z$.

One can get this with:

```
outfile.add_atoms([("C", [0, 0, 0])])

outfile.add_orbital_at_atom("s", 0, coeff=0.5)
outfile.add_orbital_at_atom("p_x", 0, coeff=0.5)
outfile.add_orbital_at_atom("p_y", 0, coeff=0.5)
outfile.add_orbital_at_atom("p_z", 0, coeff=0.5)
```

See examples/sp3_hybridization/ for more examples.

Using the same `coeff` parameter, you can also get molecular bonding and antibonding orbitals. For example, the antibonding molecular orbital of hydrogen molecule is: $\varphi_{ABO} = \frac{1}{\sqrt{2}}s_A - \frac{1}{\sqrt{2}}s_B$:

```
outfile.add_atoms([
    ('H', [0, 0, 0]),
    ('H', [0.737, 0, 0])
])
```

```
outfile.add_orbital_at_atom('s', 0, coeff =   0.707)
outfile.add_orbital_at_atom('s', 1, coeff = - 0.707)
```

See also examples/H2_molecule/.

Step 4: Write the file

Just call `.write_data()` method to write the .xsf file with the structure and the orbitals. There is optional parameter `squared=False` for this function. If `squared=True`, the value of the orbital is squared. This is useful for visualizing the density of the orbital.

Example:

```
outfile.write_data()
```

Visualizing electron and hole states from DFT+U occupation matrices

In DFT+U calculations, the local electronic state of a correlated atomic shell is commonly described by its Hubbard occupation matrix. This matrix contains information about how the localized atomic-like orbitals are occupied for a given atom, angular-momentum channel, and spin channel.

Quantum ESPRESSO prints the eigenvalues and eigenvectors of these occupation matrices in the `pw.x` output file. These quantities are often sufficient to identify which local orbital states are occupied or empty, but the corresponding spatial shape may be difficult to infer directly from the numerical coefficients.

This situation commonly occurs in systems with orbital ordering, charge disproportionation, Jahn–Teller distortions, or reduced local symmetry, where the relevant electron or hole state is a non-trivial linear combination of cubic harmonics.

The `orbitals_from_qe` method reads the occupation-matrix eigenvalues and eigenvectors from the Quantum ESPRESSO output file and converts them into real-space orbital visualizations for selected atoms.

For a given atom and spin channel, the Hubbard occupation matrix is represented by a matrix $\mathbf{N}$. Its eigenvalues and eigenvectors are printed in the Quantum ESPRESSO output file: $\mathbf{N}\mathbf{v}_i = \lambda_i \mathbf{v}_i$. Here, λ_i is the occupation of the local eigenstate i, while $\mathbf{v}_i$ is the corresponding eigenvector. The components v_{mi} of this eigenvector give the expansion coefficients of the local state in the basis of atomic orbitals $\phi_m(\mathbf{r})$:

$$\varphi_i(\mathbf{r}) = \sum_m v_{mi} \phi_m(\mathbf{r}).$$

For electron-state visualization, `atorvi` weights each eigenstate by its occupation:

$$\psi_i^{\mathrm{electron}}(\mathbf{r}) = \lambda_i \sum_m v_{mi}\phi_m(\mathbf{r}).$$

For hole-state visualization, the complementary unoccupied weight is used:

$$\psi_i^{\mathrm{hole}}(\mathbf{r}) = (1-\lambda_i) \sum_m v_{mi}\phi_m(\mathbf{r}).$$

If `eigenstate="all"`, all occupation-matrix eigenvectors are included in the generated orbital field:

$$\Psi(\mathbf{r}) = \sum_i w_i \sum_m v_{mi}\phi_m(\mathbf{r}),$$

with

$$w_i = \begin{cases} \lambda_i, & \text{for } \texttt{mode="electron"}, \\ 1-\lambda_i, & \text{for } \texttt{mode="hole"}. \end{cases}$$

If `eigenstate="dominant"`, only the eigenvector with the largest relevant weight is used. Thus, in `mode="electron"` the dominant state is the most occupied one, whereas in `mode="hole"` it is the most unoccupied one.

As an example, for atom 1 in KCuF3_scf.out, Quantum ESPRESSO prints the following eigenvalues and eigenvectors of the occupation matrix for one spin channel:

```
eigenvalues: 0.221  0.990  0.999  0.999  1.000

                 i=1     i=2     i=3     i=4     i=5
d_{3z^2-r^2}  -0.793  -0.609   0.000   0.000   0.000
d_{xz}        -0.000  -0.000   0.707  -0.000  -0.707
d_{yz}         0.000  -0.000  -0.707  -0.000  -0.707
d_{x^2-y^2}   -0.000   0.000   0.000   1.000  -0.000
d_{xy}        -0.609   0.793  -0.000  -0.000  -0.000
```

The first eigenstate has occupation $\lambda_1 = 0.221$. For hole visualization, its unoccupied weight is therefore $1-\lambda_1 = 0.779$, and the corresponding real-space state is constructed as

$$\psi_1^{\mathrm{hole}}(\mathbf{r}) = 0.779\left[-0.793\phi_{d_{3z^2-r^2}}(\mathbf{r}) - 0.609\phi_{d_{xy}}(\mathbf{r})\right].$$

In this way, the numerical occupation-matrix eigenvectors from the Quantum ESPRESSO output are transformed into a spatial orbital shape that can be inspected visually.

Example for LaMnO3:

```
import atorvi

outfile = atorvi.OrbitalFile("LaMnO3_orbitals.xsf")

outfile.orbitals_from_qe(
    qe_outfile="LaMnO3_scf.out",
    atoms=[5, 6, 7, 8],
    spin="up",
    mode="hole",
    eigenstate="dominant",
)

outfile.write_data(squared=False)
```

The `orbitals_from_qe` method accepts the following arguments:

```
"""
qe_outfile : str or pathlib.Path
    Path to a Quantum ESPRESSO pw.x output file.

atoms : list, optional
    Indices of atoms to analyze in the Quantum ESPRESSO output.
    If None, all Hubbard atoms for the selected angular-momentum channel are
 ↪  used.

l : int, optional
    Orbital angular momentum channel. The default is 2, corresponding to d
 ↪  orbitals.

spin : {"up", "down", "both"}, optional
    Spin channel to use. The default is "both".

mode : {"electron", "hole"}, optional
    Selects whether the eigenstates are weighted by occupied electron weight
    or by unoccupied hole weight. The default is "electron".

eigenstate : {"all", "dominant"} or int, optional
    Selects which occupation-matrix eigenstate is visualized:
    all eigenvectors, the eigenvector with the largest relevant weight,
    or a specific zero-based eigenvector index. The default is "all".
```

"""

See examples in examples/orbitals_from_qe/.

Note that this procedure does not reconstruct the full self-consistent DFT charge density. It visualizes the local atomic-orbital character encoded in the Hubbard occupation matrix, using the atomic-orbital basis implemented in `atorvi`. The resulting images should therefore be interpreted as qualitative representations of local electron or hole states.

Author

`atorvi` is developed and maintained by Dmitry Korotin. Contributions, suggestions, and feedback are welcome to help improve the project.